

서버 · AI

담당: ocyu007-cloud (추천) · 생성형 AI 로 각 라운드를 진행하기 위한 안내서

이 문서로 할 수 있는 것

① 내가 맡은 작업과 순서를 안다 ② 각 작업의 완료 기준을 안다 ③ 내가 정해야 할 것과 팀과 맞춰야 할 것을 구분한다 ④ AI 에게 어떤 순서로 무엇을 말할지 안다

내 라운드 한눈에 보기

라운드	작업 카드	비고
R0	통합 확인 (공동)	서버 배포 상태 확인
R1	C-1 좋아요 · 댓글 API	필수
R2	C-2 쪽지 AI 필터	필수
R2	C-3 AI 맞춤 탐방 추천 API	필수 · 대표 기능
R2	C-4 AI 공지 판별	AI 활용
R4	C-5 구조도 · 기획서 기술 항목	제출

내 파일 (파일 주인)

server/ , docs/api.md

다른 사람 파일을 고쳐야 하면 먼저 알리고, 그 사람을 PR 리뷰어로 지정합니다.

결정

노란 칸 = 작업자가 정할 것 (일부러 비워 둠)

연결

파란 표시 = 다른 팀원과 먼저 합의할 것

시작 전 준비

1. 저장소 — 팀 원본 저장소 `mungim5556-ui/KUCA` 를 기준으로 작업합니다. 이 가이드의 원본 문서는 저장소의 `docs/guides/` 폴더에 있습니다 (main 에 합쳐진 뒤).
2. AI 도구 — 저장소 파일을 직접 읽고 고칠 수 있는 도구를 권장합니다 (예: Claude Code 를 GitHub 저장소에 연결). 대화형 AI 만 쓴다면 관련 파일 내용을 복사해 함께 붙여 넣습니다.
3. 라운드 순서 — R0 통합 → R1 필수 기능 → R2 AI 핵심 → R3 게임성 → R4 제출 준비. R1 과 R2 는 세 사람이 동시에 진행합니다.
4. 이 문서에서 비어 있는 칸 — **결정** 표시가 붙은 칸은 일부러 비워 두었습니다. 작업자가 정하고, 저장소 문서의 같은 칸에 **결정: 내용 — 이름, 날짜, 이유** 로 적어 PR 에 함께 넣습니다.

1부. 공통 — 생성형 AI 로 작업하는 방법

역할별 가이드(`A_아트_출현규칙.md` , `B_애플리케이션.md` , `C_서버_AI.md`)를 쓰기 전에 이 문서를 먼저 읽습니다. 전체 라운드와 역할은 `../로드맵_역할분담.md` 에 있습니다.

이 가이드를 읽는 법

각 작업 카드에는 다섯 칸이 있습니다.

칸	뜻
목표 · 완료 기준	팀이 합의한 것. 바꾸려면 팀에 먼저 말합니다
지금 코드	출발점이 되는 파일. AI에게 그대로 알려 주세요
결정 작업자가 결정할 것	일부러 비워 둔 칸. 작업자가 정하고 결정: 뒤에 적습니다
연결 맞춰야 할 것	다른 사람 작업과 이어지는 약속. 정하기 전에 상대와 합의합니다
AI 진행 순서	AI에게 시킬 순서와 예시 문장

결정 칸은 아래처럼 채우고 같은 PR에 넣습니다. 나중에 AI가 이 기록을 읽고 일관되게 작업합니다.

- 몇 개의 시간대로 나눌지 결정: 4개 (아침/낮/저녁/밤) — 최원준, 10/10. 이유: 수업 시간표와 맞추기 쉬움

작업 한 건의 흐름

1. 카드 고르기 — 자기 역할 가이드에서 이번 라운드 카드를 고릅니다.
2. 브랜치 — 최신 main에서 `feature/<이름>-<기능>` 브랜치를 만듭니다.
3. 결정 먼저 — **결정** 칸을 AI와 상의해 정하고 문서에 적습니다. **연결** 칸은 상대방에게 확인받습니다.
4. 설계 → 구현 — AI에게 바로 코드를 쓰게 하지 말고, 먼저 계획을 받고 확인한 뒤 진행합니다.
5. 확인 — 카드의 "확인 방법"을 직접 해 봅니다. Unity는 에디터 Play, 가능하면 폰.
6. PR — main으로 PR을 열고, 파일 주인이 다른 파일을 건드렸다면 그 사람을 리뷰어로 지정합니다.

AI에게 줄 문장 (복사해서 쓰기)

세션 시작

이 저장소는 경희대 국제캠퍼스 산책·수집 게임 KUCA입니다 (Unity 6 URP 앱 + `server/` ASP.NET 메모 서버, Azure 배포). `docs/로드맵_역할분담.md`, `docs/guides/00_공통_AI작업가이드.md`, `docs/guides/〈내 역할〉.md` 를 먼저 읽으세요. 이번 작업은 카드 "<카드 번호와 이름>" 입니다. 아직 코드는 쓰지 말고, 관련 파일을 읽은 뒤 (1) 지금 구조 요약 (2) **결정** 결정할 것마다 선택지 2~3개와 장단점 (3) 구현 계획 을 알려 주세요.

결정 후 구현

결정은 이렇게 했습니다: <결정 내용>. 가이드의 해당 **결정** 칸에 기록하고, 계획대로 구현하세요. 내 파일 주인 범위 밖의 파일은 꼭 필요할 때만 최소로 고치고, 고친 이유를 알려 주세요. 끝나면 내가 직접 확인할 방법(눌러 볼 순서)을 알려 주세요.

리뷰 받기

main 과 비교한 이번 브랜치 변경을 검토해 주세요. 버그, 다른 팀원 파일과 충돌할 곳, 빠진 예외 처리(네트워크 실패, 위치 권한 없음 등)를 찾아 주세요.

막혔을 때

<증상>이 생깁니다. 재현 순서: <순서>. 에러 로그: <로그>. 추측으로 고치지 말고 원인부터 찾아 주세요.

AI에게 맡길 때 지킬 것

- 한 번에 한 카드. 여러 기능을 한 PR에 섞지 않습니다.
- Unity 씬(`SampleScene.unity`)은 AI가 직접 고치지 않게 합니다. 오브젝트는 코드에서 만들거나, 씬 수정 이 필요하면 사람이 에디터에서 합니다.
- `.meta` 파일은 Unity가 만든 것을 그대로 커밋합니다. AI가 임의로 만들면 GUID가 꼬입니다.
- API 키·비밀번호는 저장소에 넣지 않습니다. 서버 키는 Azure 앱 설정 / 로컬은 `dotnet user-secrets` . 앱 (Unity)에는 AI 키를 넣지 않고 항상 서버를 거칩니다.
- AI가 "됐다"고 해도 직접 돌려 봅니다. 이 프로젝트는 AI가 Unity를 실행해 볼 수 없는 환경이 많습니다.
- 기획서와 다른 판단을 하게 되면 PR 설명에 적습니다 (발표 때 기획서와 맞춰야 하므로).

역할 사이 약속 (**연결** **모음**)

여기 있는 약속은 정해지면 이 표에 적습니다. 한쪽이 바뀌면 다른 쪽이 깨지는 것들입니다.

약속	주는 쪽 → 쓰는 쪽	정할 것	결정
출현 조건 데이터	A → B (도감 "언제 나와요" 힌트)	파일 위치·형식	
희귀 색 개체 표시	A → B (<code>GameProgress.Caught</code> 필드)	필드 이름·값	
좋아요·댓글 API	C → B	<code>docs/api.md</code> 에 추가	
쪽지 필터 거절 응답	C → B	상태 코드·사용자에게 보일 문구	
탐방 추천 API	C → B	입력·출력 JSON	
공지 API	C → B	공지 ↔ 장소 ID 연결 형식	
날씨 정보	C 또는 A → A	앱에서 직접 vs 서버 경유	
퀘스트 보상	A(액세서리) → B(지급)	아이템 id 목록	

API는 항상 `docs/api.md` 를 먼저 고친 PR로 합의하고, 그다음 서버·앱을 구현합니다.

2부. 팀원 C 작업 카드

파일 주인: `server/` , `docs/api.md`

먼저 읽기: `00_공통_AI작업가이드.md` , `server/README.md` , `../api.md`

지금 상태 요약

- ASP.NET Core(.NET 10) + SQLite, Azure App Service 배포 (`server/deploy-azure.sh`).
- API: 건물 목록·조회, 쪽지 목록·작성(사진)·조회·삭제. 테스트는 `server/KucaMemoServer.Tests` .
- AI 를 쓰는 기능은 아직 하나도 없습니다. 기획서는 AI 활용을 핵심으로 내세우므로 R2 가 가장 중요합니다.
- 기획서에는 Firebase 로 적혀 있지만 실제로는 Azure 입니다 → 기획서 수정 필요 (R4).

AI 기능 공통 원칙

- 앱은 AI 를 직접 부르지 않고 항상 이 서버를 거칩니다 (키 보호, 비용 통제, 모델 교체 쉬움).
- 키는 Azure 앱 설정 / 로컬은 `dotnet user-secrets` . 저장소에 넣지 않습니다.
- AI 가 실패하거나 느릴 때의 대체 동작을 반드시 둡니다 (규칙 기반 결과, 시간 초과).
- AI 응답은 자유 문장이 아니라 정해진 JSON 형식으로 받게 하고, 서버에서 형식을 검사합니다.
- 기획서 "품질 관리": 공지는 원문 링크를 반드시 함께 돌려줍니다.

결정 공통으로 결정할 것

- 사용할 AI 서비스와 모델 (Gemini / Claude / 기타 — 무료 한도, 한국어 품질, 응답 속도 비교)

결정 :

- 하루 호출 상한, 같은 질문 결과 재사용(캐시) 여부

결정 :

C-1. 좋아요 · 댓글 API (R1, 필수)

완료 기준: `docs/api.md` 에 명세가 있고, 서버 구현과 테스트가 통과한다. B 의 앱에서 실제로 동작한다.

지금 코드: `Endpoints/MemoEndpoints.cs` , `Services/MemoStore.cs` , `Models/Memo*`

결정 작업자가 결정할 것

- 사용자 구분 방법 (지금처럼 기기 ID / 닉네임 / 학교 인증 후 계정) — 중복 좋아요 방지와 연결됨

결정 :

- 댓글 삭제 권한 (본인만 / 쪽지 작성자도)

결정 :

- 쪽지 목록에 좋아요 수·댓글 수를 같이 줄지

결정 :

연결 맞춰야 할 것: 명세 → B. 명세 PR 을 먼저 올리고 B 확인 후 구현.

AI 진행 순서: "docs/api.md 의 기존 규칙(상태 코드, 오류 형식, 페이지 나누기)을 따라 좋아요·댓글 명세 초안을 써주세요" → B 와 합의 → 구현 → "명세의 규칙마다 테스트를 추가해 주세요".

C-2. 쪽지 AI 필터 (R2, 필수)

목표: 기획서 "쪽지 내용 → 부적절한 내용 필터링".

완료 기준: 쪽지·댓글 작성 시 서버가 AI 로 판별하고, 부적절하면 저장하지 않고 이유를 돌려준다. AI 장애 시에도 작성 자체가 막히지 않는다.

결정 작업자가 결정할 것

- "부적절"의 기준 (욕설, 혐오, 개인정보 노출, 광고, 수업·시험 정보 유출 등 무엇을 포함할지)

결정 :

- 사진도 검사할지 (비용·속도 증가)

결정 :

- AI 장애 시 정책 (일단 저장 후 나중 검사 / 단어 목록만으로 검사 / 거절)

결정 :

- 애매한 경우 처리 (거절 / 숨김 후 검토 대기)

결정 :

연결 맞춰야 할 것: 거절 응답의 상태 코드와 사용자에게 보일 문구 → B

확인 방법: 정상·욕설·전화번호 포함·광고 문장 등 예시 묶음을 테스트로 만들어 기대 결과와 비교.

C-3. AI 맞춤 탐방 추천 API (R2, 필수 — 대표 기능)

목표: 기획서 "공강 시간과 현재 위치에 맞춰 안 가 본 장소로 가는 짧은 루트", "취향, 날씨, 계절감을 분석해 오늘 걷기 좋은 숨은 명소와 산책 코스를 큐레이션."

완료 기준: 앱이 보낸 정보로 장소 3~4곳의 순서와 장소별 한 줄 이유를 돌려준다. 실제 걸을 수 있는 거리·시간 안이다. AI 실패 시 규칙 기반 추천을 돌려준다.

지금 자료: `server/KucaMemoServer/Data/buildings.json` (건물 632개, 좌표), 앱의 경희스팟 목록 (`Memo/KyungHeeSpots.cs`), 캐릭터 랜드마크 구역(`CreatureLibrary.GoldZones`)

결정 작업자가 결정할 것

- 입력에 무엇을 넣을지 (현재 위치, 남은 시간, 방문 기록, 잡은 캐릭터, 취향, 날씨·계절)

결정 :

- 거리·시간 계산을 AI 에게 맡길지, 서버가 후보를 거리로 걸러서 AI 는 고르기·설명만 할지 (후자 권장: 엉뚱한 거리 방지)

결정 :

- 추천 후보 장소 목록 (632개 건물 전체 / 경희스팟·랜드마크만 / 직접 고른 숨은 명소 목록)

결정 :

- 숨은 명소에 대한 설명 자료를 누가 어떻게 모을지 (AI 가 학교를 잘 모르므로 직접 쓴 짧은 소개가 품질을 좌우)

결정 :

- 개인정보: 방문 기록을 서버에 저장할지, 요청 때만 보내고 버릴지 (기획서 7번과 맞춤)

결정 :

연결 맞춰야 할 것: 입력·출력 JSON → B. 출현 조건(A-1)을 추천 이유에 쓸지("지금 가면 밤에만 나오는 날다람쥐를 만날 수 있어요") → A

AI 진행 순서: 명세 → 규칙 기반 추천(거리 + 미방문)을 먼저 만들어 대체 동작 확보 → 그 위에 AI 고르기·설명 추가 → 같은 입력으로 여러 번 돌려 결과가 안정적인지 확인.

C-4. AI 공지 판별 (R2)

목표: 기획서 "학교 공지사항을 올라오는 즉시 AI 가 판별하여 각 장소에 맞는 공지사항을 입력", "공지 사항의 링크를 무조건 첨부."

완료 기준: 공지 원문(제목·본문·링크)을 넣으면 관련 장소 ID 와 짧은 요약이 붙어 저장되고, 앱이 장소별 공지를 받아 갈 수 있다.

결정 작업자가 결정할 것

- 공지 수집 방법 (학교 사이트 자동 수집 / 운영자가 붙여 넣기) — 자동 수집은 사이트 이용 규칙 확인 필요

결정 :

- 장소와 무관한 공지 처리 (버림 / 전체 공지로)

결정 :

- 지금 앱의 `Data/Notices.json` 을 서버 공지로 대체할지

결정 :

연결 맞춰야 할 것: 공지 API 와 장소 ID 형식 → B

C-5. 구조도 · 기획서 기술 항목 (R4)

완료 기준: 앱 ↔ Azure 서버 ↔ AI 서비스 ↔ (날씨 API) 연결 그림 한 장, 기획서 5·7·8번 칸(AI 활용, 데이터·개인정보, 기술 구성) 갱신.

결정 작업자가 결정할 것

- 비용 추정에 쓸 가정 (사용자 수, 1인 하루 호출 수)

결정 :